

Q.1

int[] arr = {1, 2, 2, 3, 3, 3, 4, 4}

Page No.

Date

HashMap<Integer, Integer> hm = new HashMap<>();

for (int i = 0; i < arr.length; i++) {

int num = arr[i];

hm.put(num, hm.getOrDefault(num, 0) + 1);

}

int maxFreq = 0;

{ for (int freq : hm.values()) {

if (freq > maxFreq) {

maxFreq = freq;

}

int secondMaxFreq = 0;

for (int freq : hm.values()) {

if (freq > secondMaxFreq && freq < maxFreq) {

secondMaxFreq = freq;

}

for (hm.entrySet().entrySet().iterator().next()) {

if (entry.getValue() == secondMaxFreq) {

sup(entry.getKey());

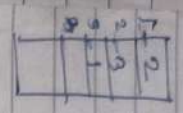
}

2 or 4

int[] arr = {1,1,2,2,2,3};
 int k = 2

hash < Integer, Integer > hm = new HashMap<>();

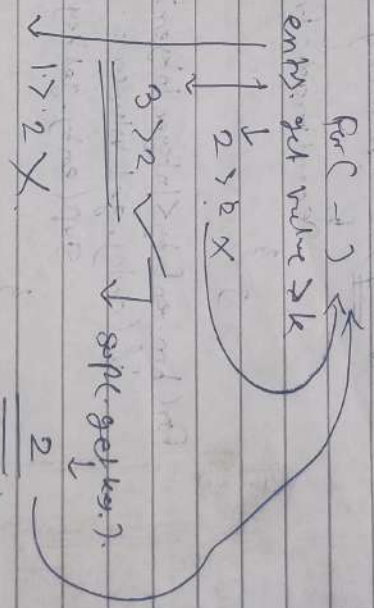
```
for (int num : arr) {
    hm.put(num, hm.getOrDefault(num, 0) + 1);
}
```



```
for (hm.Entry<Integer, Integer> entry : hm.entrySet()) {
    if (hm.get(entry.getKey()) > k) {
        hm.remove(entry.getKey());
    }
}
```

o/p > 2

1 → 2
 2 → 3
 3 → 1



Q3. ~~exactly~~ frequency element

int

int arr[] = {1,1,2,2,2,3}

int k = 2

HashMap<Integer, Integer> hm = new HashMap<>();

```
for (int num : arr) {
```

```
    hm.put(num, hm.getOrDefault(num, 0) + 1);
}
```

2

for(hm.Entry < Integer, Integer> entry : hm.entrySet()) {

if (entry.getValue() == k) {

sup(entry.getKey());

}

0/p > 1

Q.4

only once, no repeating elements

→ int arr[] = {1, 1, 2, 2, 3, 3}

hashmap < Integer, Integer> hm = new HashMap<>();

for(int num : arr) {

hm.put(num, hm.getOrDefault(num, 0) + 1);

}

for(hm.Entry < Integer, Integer> entry : hm.entrySet()) {

if (entry.get^{get}Value() == 1) {

sup(entry.getKey());

}

Q.5

return array and duplicate elements remove.

int arr = {1, 2, 2, 2, 3}

int[] result = f(arr);

sup(

int[] newArr = f(arr);

sup(newArr);

}

1	1
2	3
3	1

```
int[] f(int[] arr) {
```

```
    HashMap<Integer, Integer> hm = new HashMap<>();
```

```
    for (int num: arr) {
```

```
        hm.put(num, hm.getOrDefault(num, 0) + 1);
```

```
    }
```

```
    int count = 0;
```

```
    for (int num: hm.values()) {
```

```
        if (hm.get(num) > 1) {
```

```
            if (num == 1) {
```

```
                count++;
```

```
            }
```

```
    }
    int[] newarr = new int[count];
```

```
    int index = 0;
```

```
    for (int num: arr) {
```

```
        if (hm.get(num) < 1 || hm.get(num) > 1) {
```

```
            if (hm.get(num) == 1) {
```

```
                newarr[index] = num;
                index++;
```

```
            }
```

```
        }
```

```
    }
```

```
    return newarr;
```

Q6 freq character

psume

character, integer

HashMap < Integer, Integer > hm = new HashMap < Integer, Integer >();

for (char ch : hm) {

if (hm.containsKey(ch)) {

hm.put(ch, hm.get(ch) + 1);

}

else { hm.put(ch, 1);

}

char ch = 'a';

~~for (char ch : hm)~~ in max = 0;

for (char key : hm.entrySet())

if (hm.get(key) < max) { hm.put(key, max);

if (hm.get(key) > max) {

max = hm.get(key);

char = hm.get(key);

}

sop(char);

get value of value

}

entry.getKey() of value of key character.

a	1
b	2
c	1
d	1

```
int min = Integer.MAX_VALUE;
int second = Integer.MAX_VALUE;
```

Q. Second least element

psvmc}

Hash -

arr = {1, 1, 2, 2, 2, 3, 3}

```
for (int num: arr) {
```

```
    hmap.put(num, hmap.getOrDefault(num, 0) + 1);
```

}

```
int first = 0; int second = Integer.MAX_VALUE;
```

```
for (hmap.Entry <Integer, Integer> entry : hmap.entrySet()) {
```

```
    if (entry.getValue() > first)
```

```
        first = entry.getValue();
```

```
    if (entry.getValue() < min)
```

```
        min = entry.getValue();
```

```
    first = min;
```

```
    min = entry.getValue();
```

```
    second = min;
```

}

```
if (entry.getValue() < second && entry.getValue() > first)
```

```
    second = entry.getValue();
```

}

```
for (hmap.Entry <Integer, Integer> entry : hmap.entrySet()) {
```

```
    if (entry.getValue() < second)
```

```
        second = entry.getValue();
```

}

}

psvmc}

String S = "aagacba";

```
HashSet<Character> set = new HashSet<>();
```

```
for (int i = 0; i < S.length(); i++) {
```

```
    char ch = S.charAt(i);
```

```
    if (ch > 'a' && ch <= 'z') {
```

```
        set.add(ch);
```

}

}

}

```
if (set.size() == 26) {
```

```
    sop("True");
```

}

```
else { sop("False");
```

}

}

Missing elements Peto two arrays

arr1 = {1, 2, 2, 2, 3, 3, 7}

arr2 = {1, 1, 2, 2, 2, 3}

```
HashMap<Integer, Integer> hmap = new HashMap<>();
```

```
HashMap<Integer, Integer> hmap2 = new HashMap<>();
```

```
for (int n: arr1) {
```

```
    hmap.put(n, hmap.getOrDefault(n, 0) + 1);
```

```
    hmap.put(n, hmap.getOrDefault(n, 0) + 1);
```

}

```
for (int n: arr2) {
```

```
    hmap2.put(n, hmap2.getOrDefault(n, 0) + 1);
```

```
    hmap2.put(n, hmap2.getOrDefault(n, 0) + 1);
```

priority queue

```

if (HashMap.containsKey(key)) {
    HashMap.put(key, HashMap.get(key) + 1);
} else {
    HashMap.put(key, 1);
}

```

```

for (int num : arr) {
    if (HashMap.containsKey(num)) {
        HashMap.put(num, HashMap.get(num) + 1);
    } else {
        HashMap.put(num, 1);
    }
}

```

```

if (2 > 0) {
    HashMap.put(2, 1);
}

```

```

if (1 > 0) {
    HashMap.put(1, 1);
}

```

```

if (1 > 0) {
    // ...
}

```

```

else {
    HashMap.put(num, 1);
}

```

2 2 1

1 2 3 1

HashMap.containsKey(key) ?
arr1 {1, 2, 2, 3, 3} arr2 {2, 2, 4, 3}

for (int num : arr1)

HashMap.put(num, HashMap.getOrDefault(num, 0) + 1);

}

arr1 [1, 2, 2, 3]

for (int n : arr2)

arr2 [1, 2, 4]

if (HashMap.containsKey(n))

HashMap.put(n, HashMap.getOrDefault(n, 0) + 1);

HashMap.put(n, HashMap.get(n) + 1);

Sop(n);

}

1 1 2 1

else

if (0

arr1 = {1, 2, 2, 3}

arr2 = {2, 2, 4, 3}

int[] arr = new int[arr1.length + 1];

int max = 0;

for (int i : arr1)

if (max < arr1[i])

max = arr1[i];

for (int i : arr2)

freq[i]++;

}

for (int i : arr2)

if (freq[i] > 0)

freq[i]--;

Sop(n);

}

Anagram → length same, character same
 Count same, character same
 String s1 = "abcdeh" for character frequency same
 String s2 = "aehcd" anagram

```
if (s1.length() == s2.length()) {
    // Not anagram
}
```

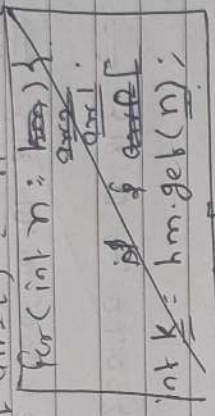
```
HashMap
for (char ch = s1) {
    hm.put(ch, hm.get(ch) + 1);
    hm.put(ch, 1);
}
else {
    hm.put(ch, 1);
}
```

```
for (char ch = s2) {
    if (hm.containsKey(ch)) {
        hm.put(ch, hm.get(ch) - 1);
    }
    else {
        // Not anagram
    }
}
```

Q13 Replace element with key
 → arr1 [1, 2, 2, 3].

hashmap < int, int > hm

```
for (int n : arr1) {
    hm.put(n, hm.getOrDefault(n, 0) + 1);
}
int arr2[] = new int[arr1.length];
```



```
for (int i = 0; i < arr1.length; i++) {
    int k = hm.get(arr1[i]);
    arr2[i] = k;
}
```